

社区团购管理系统

软件工程项目设计说明书

小组名称： 组长：陈小庆 成员：王波，魏伊阳，杨洋

2026年5月

目 录

1. 文档概述.....	3
1.1 文档目的.....	3
1.2 项目背景.....	3
1.3 文档范围.....	3
1.4 术语与定义.....	3
1.5 参考资料.....	3
2. 项目概述.....	4
2.1 项目名称.....	4
2.2 项目目标.....	4
2.3 适用人群.....	4
2.4 项目规模.....	4
3. 总体架构设计.....	5
3.1 架构选型.....	5
3.2 总体架构图.....	5
3.3 技术栈细化（与需求文档一致，补充设计层面细节）	7
4. 模块详细设计.....	8
4.1 模块划分（与需求文档功能模块一致）	8
4.2 模块依赖关系.....	8
4.3 核心模块详细设计.....	8
4.3.1 用户模块.....	8
4.3.2 管理员模块.....	11
4.3.3 商家模块.....	13
5. 数据库设计.....	15
5.1 数据库设计原则.....	15
5.2 E-R 图设计.....	15
5.3 数据表设计.....	17
5.4 索引设计.....	19
5.5 数据库脚本.....	19
6. 接口设计.....	25
6.1 接口设计原则.....	25
6.2 接口通用格式.....	25
6.2.1 请求格式.....	25
6.2.2 返回格式.....	25
6.3 核心接口详细设计.....	26
6.3.1 用户管理模块接口.....	26
6.3.2 商品管理模块接口.....	27
7. 界面详细设计.....	33

7.1 界面设计原则.....	33
7.2 核心界面详细设计（按页面分类）	33
8. 开发规范.....	39
8.1 编码规范.....	39
8.2 文档规范.....	40
8.3 版本控制规范.....	40
9. 附录.....	40
9.1 团队信息.....	40
9.2 其他说明.....	40

软件工程项目设计说明书

1. 文档概述

1.1 文档目的

本文档基于《社区团购管理系统》需求规格说明书，明确系统的总体架构、模块设计、数据库设计、接口设计等核心内容，为编码实现、测试验收提供标准化设计依据，确保开发团队按统一规范完成项目开发，同时培养实训小组的系统设计与工程化思维。

1.2 项目背景

本项目为解决传统社区商品购买信息获取困难、活动公告滞后、交易效率低下等问题，面向社区居民、商家及平台管理者打造线上团购平台，是本科生软件工程项目开发综合实训成果，旨在通过系统设计与实现，提升团队工程实践、需求落地与架构设计能力。

1.3 文档范围

本文档涵盖系统总体架构设计、模块详细设计、数据库设计、接口设计、界面原型设计、技术选型细化、开发规范等内容，不包含具体编码实现细节、测试用例设计及部署方案。

1.4 术语与定义

术语 / 缩写	定义 / 说明
MVC	模型 - 视图 - 控制器，系统采用的核心分层设计模式
RBAC	基于角色的访问控制，用于实现用户、商家、管理员权限隔离
API	应用程序编程接口，前后端交互、模块间数据通信的标准通道
Spring Boot	后端核心开发框架，用于快速构建 Java 后端服务

表 1-1 术语与定义

1.5 参考资料

1. 《软件工程导论》（张海藩，清华大学出版社）
2. Spring Boot 官方技术文档
3. MySQL 数据库官方开发文档

2. 项目概述

2.1 项目名称

社区团购管理系统

2.2 项目目标

1. 功能目标：实现用户注册登录、商品信息浏览与查询、购物车管理、在线下单、拼团购买、订单跟踪、留言反馈等功能；商家能够对商品信息、订单状态、活动公告进行维护管理；管理员实现对用户、商家、商品分类、商品信息、活动公告、系统配置等全局管理。满足社区居民便捷团购和商家高效运营的核心需求。

2. 性能目标：系统页面响应时间 ≤ 3 秒；支持并发用户数 ≥ 50 人；系统运行稳定，无界面模糊、字体不清等问题；数据更新及时准确，保证信息一致性和安全性。

3. 实训目标：通过需求分析，掌握需求梳理、文档编写方法，为后续设计、编码、测试奠定基础；培养工程化思维，提升团队协作与项目管理能力。

2.3 适用人群

1. 普通用户：社区内居民，通过系统注册登录后浏览商品、参与拼团、下单购买、管理个人资料、查看订单状态、发表留言和评价。

2. 商家：商品供应商或社区团长，登录后台管理商品信息（上架、下架、编辑）、处理订单（发货、更新物流）、发布活动公告。

3. 系统管理员：负责系统全局维护，包括用户与商家账号管理、商品分类设置、商品信息审核、活动公告审核、系统参数配置（如轮播图、公告等），确保系统正常运行。

2.4 项目规模

☐ 小型项目（核心功能 ≤ 10 个，适合4-6人小组2-4周完成） ☒ 中型项目（核心功能10-20个，适合4-6人小组3-4周完成）

3. 总体架构设计

3.1 架构选型

本系统采用前后端分离的 B/S 架构 + 微服务分层架构，架构选择依据：

贴合 BIM 项目管理系统的业务特性：系统需同时支持 Web 端、小程序、APP 多端访问，前后端分离架构可实现多端复用同一套后端服务，适配展示层的多端入口；

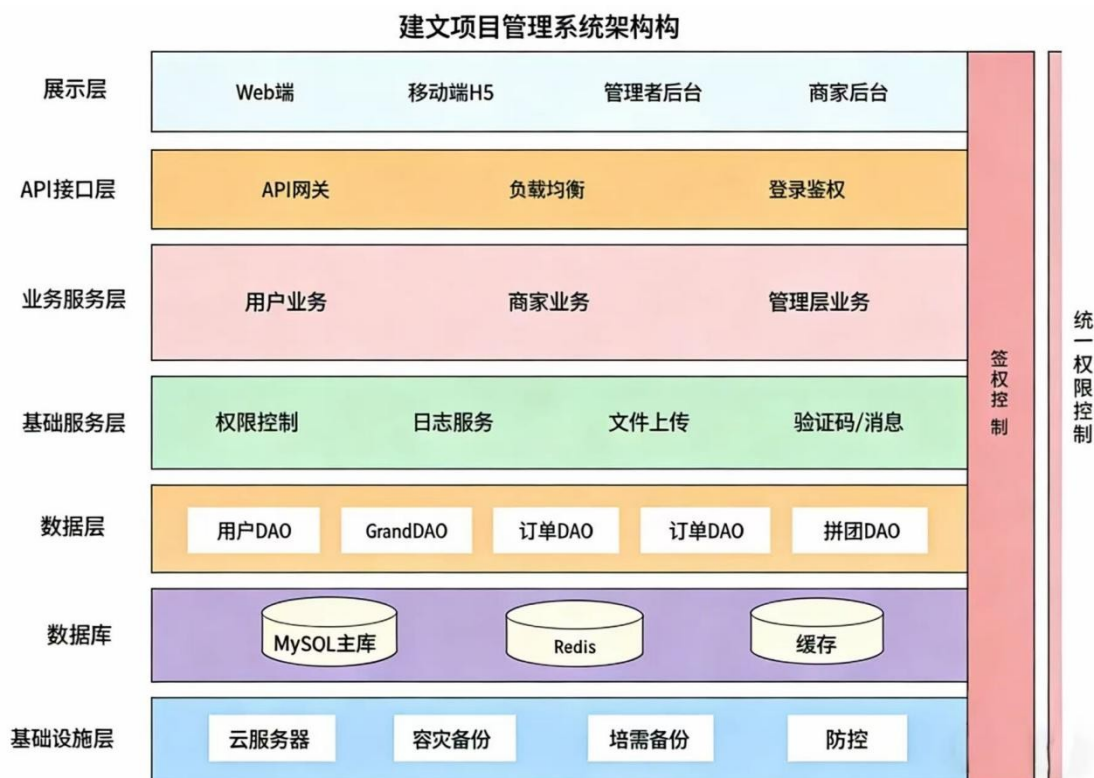
便于团队分工协作开发：前端团队专注界面交互与多端适配，后端团队专注业务逻辑与数据处理，开发流程解耦，提升开发效率；

可扩展性与维护性强：分层架构将业务服务、基础服务、数据层解耦，后续新增 BIM 模型管理、施工管理等模块时，无需改动核心架构，符合项目迭代需求；

适配企业级应用标准：支持 API 网关、负载均衡、统一权限控制等企业级特性，满足 BIM 项目管理系统的高并发、高可用、数据安全要求；

技术栈成熟度高：前后端分离架构是当前工程管理系统的主流选型，生态完善，便于解决开发过程中的技术问题。

3.2 总体架构图



架构说明：

-展示层：系统的用户交互入口，覆盖 Web 端、移动端 H5、管理员后台、商家后台四大终端，负责页面渲染、用户操作交互、前端逻辑处理，将用户请求转发至 API 接口层，同时接收后端响应并展示给用户，适配普通用户、商家、管理员三类角色的不同使用场景。

-API 接口层：前后端通信的核心桥梁，通过 API 网关实现请求路由、协议转换，搭配负载均衡实现多实例服务的流量分发，同时集成登录鉴权能力，统一拦截未授权请求，规范前后端交互格式，保障系统访问安全。

-业务服务层：系统的核心业务逻辑层，承载社区团购的全流程业务，分为三大业务域：

-用户业务：实现用户注册、购物、订单、拼团等 C 端核心功能；

-商家业务：实现商品管理、订单管理、店铺管理等商家端核心功能；

-管理员业务：实现商品审核、系统配置、平台管理等后台管理功能；

-基础服务层：为业务服务层提供通用支撑能力，包含权限控制、日志服务、文件上传、验证码 / 消息等通用模块，避免业务模块重复开发，提升系统复用性，同时为全系统提供统一的工具能力支撑。

-数据访问层（DAO 层）：负责与数据层的交互，封装了用户、商品、订单、拼团四大核心模块的数据访问逻辑，实现数据库操作的统一封装，隔离业务逻辑与数据存储细节，提升代码可维护性。

-数据层：系统的数据存储核心，采用 MySQL 主库作为关系型数据库，存储业务核心数据（用户、商品、订单等），搭配 Redis 缓存热点数据（如商品列表、用户信息），降低数据库压力，提升接口响应速度，保障数据访问的高效性与一致性。

-基础设施层：系统的底层运行保障，包含云服务器、容灾备份、防火墙，为上层所有层级提供稳定、安全、可扩展的运行环境，保障系统 7×24 小时高可用，同时通过容灾备份保障数据安全。

-前端层：负责用户交互，适配多端展示，是实训中界面开发、交互逻辑实现的核心模块，对应考核中“阶段性任务”“项目终期提交”的界面相关要求。

-接口层：作为前后端通信桥梁，负责请求路由、鉴权和异常处理，规范接口调用，贴合实训中“开发规范”的考核要点。

-业务逻辑层：核心层级，实现企业级应用的核心业务功能、规则控制和权限管理，是实训中“技术应用”“问题解决能力”的主要考核载体。

-数据访问层：负责数据持久化操作，整合 ORM 框架和缓存，提升数据访问效率，对应实训中数据库操作、性能优化相关要求。

- 数据库层：存储系统核心数据，支持关系型和非关系型数据库，包含数据备份功能，保障数据安全，贴合企业级应用开发实际需求

3.3 技术栈细化（与需求文档一致，补充设计层面细节）

技术类别	具体技术 / 工具	设计层面说明
前端	Vue3 + Element Plus + Vant（多端适配）	1. 采用组件化开发，封装通用表单、表格、弹窗组件，提升页面复用性与开发效率；2. 基于 Vue Router 实现多端路由管理，适配 Web 端、H5、商家 / 管理员后台的页面跳转逻辑；3. 搭配 Axios 封装统一请求拦截器，实现 API 请求统一管理、token 鉴权、异常处理；4. 适配移动端响应式布局，保障 H5 端在不同设备上的展示效果一致。
后端	Spring Boot + MyBatis-Plus + Spring Security	1. 采用 Spring Boot 快速搭建项目，基于 MVC 分层架构，拆分 Controller、Service、Mapper 层，实现业务逻辑与数据访问的解耦；2. 基于 MyBatis-Plus 实现数据访问层封装，简化数据库操作，支持事务控制，保障订单、拼团等核心业务的数据一致性；3. 使用 Spring Security + JWT 实现统一身份认证与权限控制，适配系统多角色（普通用户、商家、管理员）的权限管理；4. 采用 Spring Cloud Gateway 作为 API 网关，实现请求路由、限流、鉴权，搭配 Nginx 实现负载均衡；5. 封装全局异常处理器、统一返回格式，规范接口返回，提升系统可维护性。
数据库	MySQL 8.0 + Redis	1. MySQL 8.0 作为主库，采用 InnoDB 引擎，支持事务、行级锁，保障订单、支付等核心业务的数据一致性；2. 合理设计联合索引、唯一索引，优化商品列表、订单列表等复杂查询的性能，适配社区团购系统的多条件查询需求；3. Redis 作为缓存中间件，缓存热点数据（如商品列表、用户信息、拼团活动），降低数据库压力，提升接口响应速度；4. 采用主从复制架构，实现读写分离，提升数据库并发处理能力。
其他工具	Git + Postman + 墨刀 + Docker	1. Git 用于代码版本控制，搭配 Gitee/GitLab 实现团队协作开发，分支管理保障代码稳定性；2. Postman 用于接口测试，验证接口请求格式、返回数据、异常处理，保障接口可用性；3. 墨刀用于原型设计，提前规划界面布局与交互逻辑，对齐前后端开发需求；4. Docker 实现容器化部署，打包项目环境，简化部署流程，适配云服务器部署。

表 3-1 技术栈细化

4. 模块详细设计

4.1 模块划分（与需求文档功能模块一致）

本项目共分为 3 个核心模块，分别为：前台用户模块、管理员模块、商家模块，各模块职责明确、接口清晰，无冗余功能。

4.2 模块依赖关系

（附：模块依赖关系图粘贴处，标注各模块之间的调用关系，如：用户管理模块为其他模块提供用户认证服务；交易管理模块依赖物品发布模块的物品数据）

依赖关系说明：

1. 管理员模块为用户模块、商家模块提供权限校验、账号审核、内容审核、系统配置等全局管理服务，是系统的核心管控模块。
2. 用户模块依赖商家模块提供的商品、活动公告数据完成购物、拼团操作，所有操作需经管理员模块权限校验。
3. 商家模块依赖用户模块的订单、留言数据完成业务处理，商品、公告需经管理员模块审核后才能展示。
4. 模块间单向依赖，无循环依赖，架构清晰稳定。

4.3 核心模块详细设计

4.3.1 用户模块

4.3.1.1 模块功能概述

实现普通用户注册、登录、个人信息管理、商品浏览 / 搜索 / 收藏 / 评论、购物车管理、下单购买、发起 / 参与拼团、订单管理、收货地址管理、留言反馈等功能，为社区居民提供完整的团购购物服务，与需求文档前台用户功能完全一致。

4.3.1.2 模块流程图

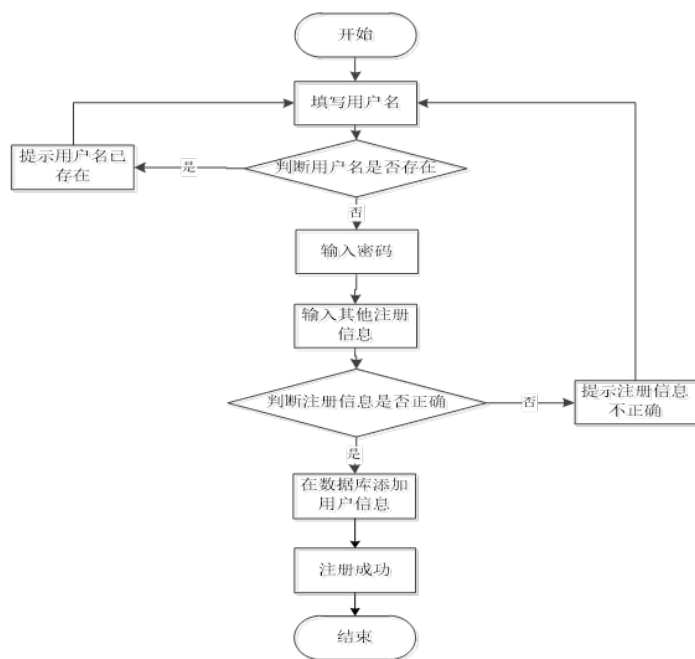


图 4-1 用户注册流程图

清晰展示用户模块的注册全流程，实现用户账号创建、信息校验、数据入库，是系统用户准入的核心流程，保障用户账号唯一性与信息合法性。

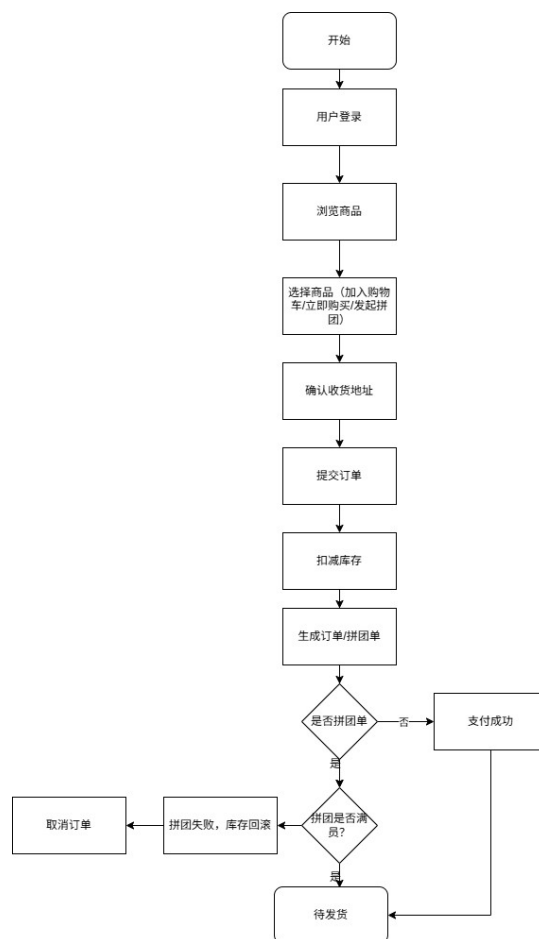


图 4-2 用户下单&拼团流程图

展示用户模块核心交易流程，覆盖普通购买与拼团两种模式，实现订单生成、库存管控，是社区团购系统的核心业务流程。

4.3.1.3 核心算法/逻辑设计

1.密码加密逻辑：采用 BCrypt 不可逆加密算法存储密码，登录时密文比对，杜绝明文泄露。

2.身份认证逻辑：基于 Token 实现登录态管理，未登录用户仅可浏览首页，无法执行交易、评论操作。

3.数据校验逻辑：注册 / 下单时校验手机号、邮箱格式，购物车数量不超过商品单限与库存。

4.拼团核心逻辑：发起拼团后预扣库存，达到指定拼团人数则拼团成功，超时未达标则拼团失败、库存回滚。

5.订单状态逻辑：待付款→待发货→待收货→已完成，未付款订单可取消，已发货订单不可取消。

4.3.1.4 核心类设计

类名	类职责	核心方法	方法说明
UserController	处理用户前端所有接口请求	register() 、 login() 、 updateInfo()	接收注册、登录、信息修改、地址管理请求，返回结果
UserService	用户核心业务逻辑实现	checkUserExist() 、 encryptPassword()	校验用户唯一性、密码加密、Token生成、登录校验
UserDao	用户数据持久化操作	insertUser() 、 selectUserByAccount()	执行用户数据增删改查，对接MySQL数据库
CartController	购物车功能接口处理	addCart() 、 updateCartNum()	处理商品加入购物车、数量修改、删除、清空
OrderController	订单 & 拼团接口处理	createOrder() 、 joinGroup()	处理下单、发起拼团、参与拼团、订单查询
CollectController	商品收藏接口处理	addCollect() 、 cancelCollect()	处理商品收藏、取消收藏、收藏列表查询

表 4-1 用户模块核心类设计

4.3.2 管理员模块

4.3.2.1 模块功能概述

实现管理员登录、用户管理、商家管理、商品分类管理、商品信息审核 / 管理、活动公告管理、系统配置（轮播图、平台公告）等功能，负责系统全局运营管控、权限分配、内容审核与数据维护，与需求文档管理员后台功能完全一致。

4.3.2.2 模块流程图

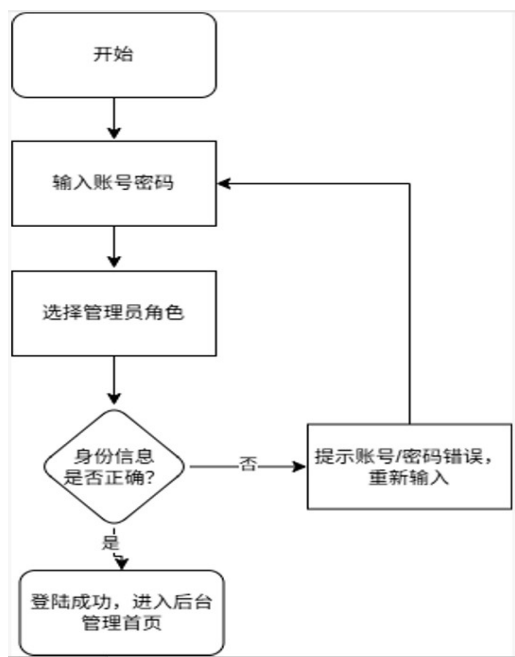


图 4-3 管理员登录流程图

明确管理员模块的登录校验流程，实现角色权限验证，保障系统后台管理的安全性，防止非管理员越权访问。

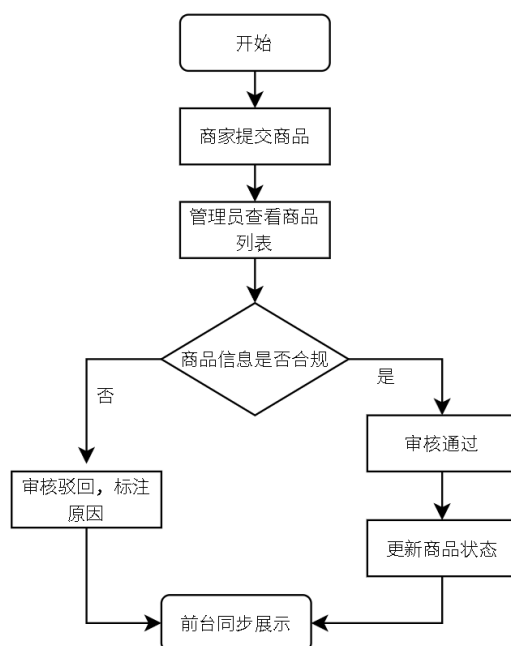


图 4-4 商品审核流程

展示管理员模块的商品审核流程，实现商品合规性校验，管控平台商品质量，是系统内容安全的核心管控流程。

4.3.2.3 核心算法/逻辑设计

- 1.RBAC 权限控制逻辑：严格区分管理员、商家、用户角色，禁止越权访问后台管理页面。
- 2.内容审核逻辑：对商品、公告、留言进行敏感词过滤，违规内容直接驳回并标注原因。
- 3.数据管控逻辑：支持用户 / 商家账号增删改查、密码重置，商品分类新增 / 修改 / 删除（无关联商品时）。
- 4.系统配置逻辑：首页轮播图、平台公告可视化配置，修改后实时同步至前台。

4.3.2.4 核心类设计

类名	类职责	核心方法	方法说明
AdminController	管理员后台接口请求处理	login() 、manageUser() 、	处理登录、用户管理、商家管理、商

		auditGoods()	品审核请求
AdminService	管理员核心业务逻辑	checkPermission()、auditContent()	权限校验、内容审核、数据统计、系统配置
AdminDao	管理员数据持久化操作	selectAdmin()、updateUserStatus()	执行管理员登录验证、用户 / 商家数据修改
CategoryController	商品分类管理接口	addCategory()、deleteCategory()	处理商品分类新增、修改、删除、查询
NoticeController	平台公告管理接口	publishNotice()、topNotice()	处理平台公告发布、编辑、删除、置顶
SystemController	系统配置接口	setCarousel()、configPlatform()	处理首页轮播图、系统参数配置

表 4-2 管理员模块核心类设计

4.3.3 商家模块

4.3.2.1 模块功能概述

实现商家注册、登录、个人信息管理、商品信息管理（新增 / 编辑 / 上下架）、订单管理（查看 / 发货 / 更新物流）、店铺活动公告管理等功能，为商品供应商提供商品运维、订单处理、店铺运营服务，与需求文档商家后台功能完全一致。

4.3.2.2 模块流程图

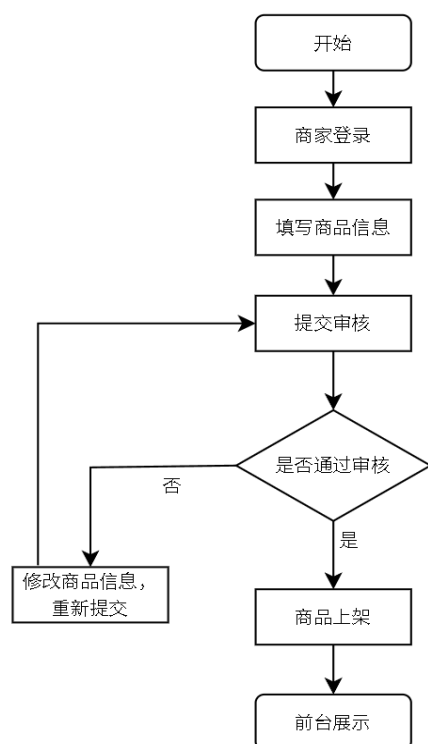


图 4-5 商家商品发布流程图

规范商家模块的商品发布流程，实现商家商品上架申请、审核联动，保障商家商品合规上架，支撑平台商品供给。

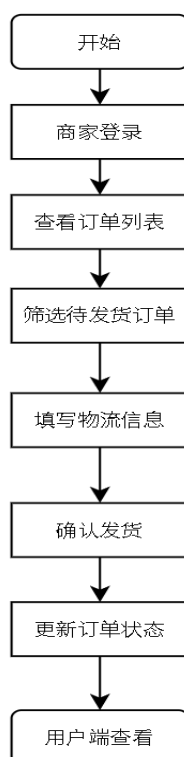


图 4-6 商家订单处理流程

明确商家模块的订单处理流程，实现订单发货、物流更新、状态同步，完成团购交易的履约环节，保障订单流程闭环。

4.3.2.3 核心算法/逻辑设计

- 1.商家认证逻辑：商家注册信息需校验完整性，账号需经管理员审核通过后才能登录使用后台功能。
- 2.商品管理逻辑：商品信息需校验价格、库存、单限合法性，上下架状态实时同步前台。
- 3.订单处理逻辑：仅可处理本店商品订单，支持发货、更新物流、取消未付款订单。

4.公告管理逻辑：仅可发布本店活动公告，内容需符合平台规范，无需管理员二次审核。

4.3.2.4 核心类设计

类名	类职责	核心方法	方法说明
MerchantController	商家后台接口处理	register() 、 login() 、 manageGoods()	处理商家注册、登录、商品管理、订单处理请求
MerchantService	商家核心业务逻辑	checkMerchantExist() 、 sendGoods()	商家信息校验、商品上下架、订单发货、物流更新
MerchantDao	商家数据持久化操作	insertMerchant() 、 updateOrderStatus()	执行商家数据增删改查、订单状态修改
MerchantGoodsController	商家商品管理接口	addGoods() 、 updateGoods()	处理商家商品新增、编辑、上下架、删除
MerchantOrderController	商家订单管理接口	getOrderList() 、 updateLogistics()	处理本店订单查询、发货、物流信息填写
MerchantNoticeController	商家公告管理接口	publishMerchantNotice()	处理店铺活动公告发布、编辑、删除

表 4-3 商家模块核心类设计

5. 数据库设计

5.1 数据库设计原则

- 1. 贴合项目需求，确保数据实体、属性完整，无冗余数据；
- 2. 遵循数据库设计范式（至少满足第三范式），减少数据冗余与更新异常；
- 3. 合理设计主键、外键，确保数据完整性与一致性；
- 4. 结合实训技术栈，选择合适的数据类型与约束条件，提升查询效率。

5.2 E-R 图设计

概念模型是对现实中的问题出现的事物的进行描述，ER 图是由实体及其关系构成

的图，通过 E-R 图可以清楚地描述系统涉及到的实体之间的相互关系。

商家信息实体体属图如下所示。

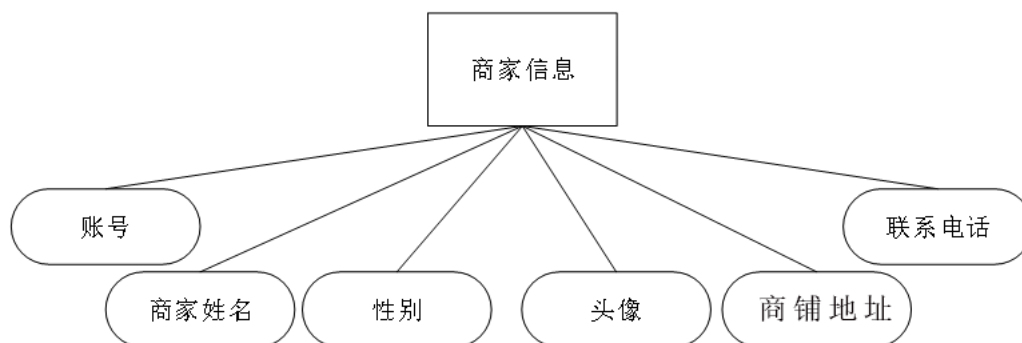


图 4-2 商家信息实体属性图

用户信息实体属图如下所示。

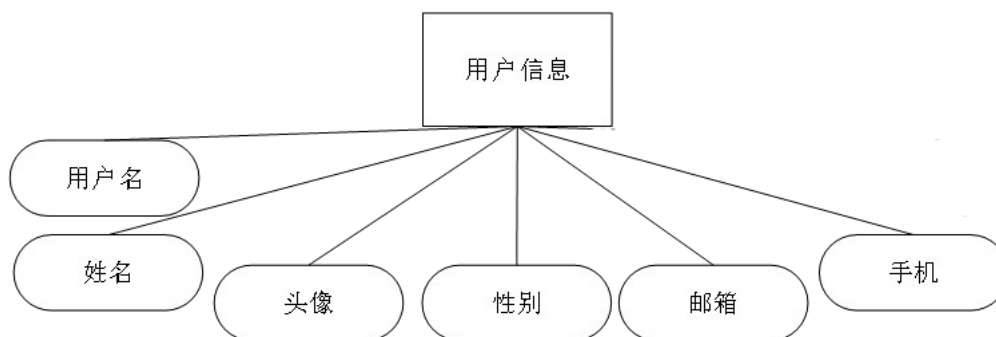


图 4-3 用户信息实体属图

商品信息实体属图如下所示。

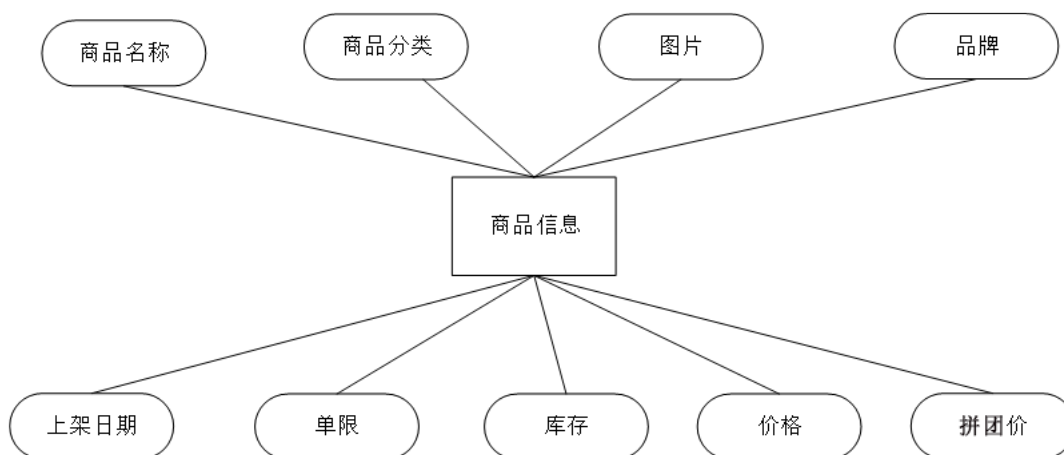


图 4-4 商品信息实体属图

E-R 图说明：本系统核心实体包含商家信息、用户信息、商品信息；其中商家信息实体与商品信息实体为一对多关系，用户信息实体与商品信息实体为多对多关系。

5.3 数据表设计

表名	表说明	主键	外键	字段详情（字段名、数据类型、长度、是否必填、默认值、备注）
user （用户表）	存储用户基本信息与权限	user_id	无	1. user_id: int (11), 必填, 自增, 主键, 用户唯一标识; 2. username: varchar (50), 必填, 唯一, 用户名; 3. password: varchar (100), 必填, 加密存储, 用户密码; 4. role: varchar (20), 必填, 默认普通用户, 用户角色（普通用户 / 管理员）; 5. create_time: datetime, 必填, 默认当前时间, 注册时间; 6. name: varchar (20), 必填, 用户真实姓名; 7. phone: varchar (11), 必填, 唯一, 用户手机号; 8. email: varchar (50), 非必填, 用户邮箱; 9. avatar: varchar (255), 非必填, 默认空, 用户头像; 10. gender: varchar (4), 非必填, 默认未知, 用户性别
merchant （商家表）	存储商家店铺信息	merchant_id	无	1. merchant_id: int (11), 必填, 自增, 主键, 商家唯一标识; 2. merchant_account: varchar (50), 必填, 唯一, 商家登录账号; 3. password: varchar (100), 必填, 加密存储, 商家密码; 4. merchant_name: varchar (50), 必填, 商铺名称; 5. phone: varchar (11), 必填, 唯一, 商家联系电话; 6. address: varchar (255), 必填, 商铺地址; 7. status: tinyint (1), 必填, 默认 0, 审核状态（0 待审核 / 1 已通过）; 8. create_time: datetime, 必填, 默认当前时间, 入驻时间
category （商品分类表）	存储商品分类信息	category_id	无	1. category_id: int (11), 必填, 自增, 主键, 分类唯一标识; 2. category_name: varchar (30), 必填, 唯一, 分类名称; 3. sort: int (11), 必填, 默认 0, 分类排序权重; 4. create_time: datetime, 必填, 默认当前时间, 创建时间
goods （商品表）	存储团购商品信息	goods_id	merchant_id、category_id	1. goods_id: int (11), 必填, 自增, 主键, 商品唯一标识; 2. goods_name: varchar (100), 必填, 商品名称; 3. category_id: int (11), 必填, 外键, 关联分类表; 4. merchant_id: int (11), 必填, 外键, 关联商家表; 5. price: decimal (10,2), 必填, 商品原价; 6. group_price: decimal (10,2), 必填, 拼团价格; 7. stock: int (11), 必填, 默认 0, 商品库存; 8. limit_num: int (11), 必填, 默认 1, 单人限购数量; 9. image: varchar (255), 必填, 商品图片; 10.

				status: tinyint (1), 必填, 默认 0, 上架状态 (0 待审核 / 1 已上架 / 2 已驳回)
address (收货地址表)	存储用户收货地址	address_id	user_id	1. address_id: int (11), 必填, 自增, 主键, 地址唯一标识; 2. user_id: int (11), 必填, 外键, 关联用户表; 3. receiver: varchar (20), 必填, 收货人姓名; 4. phone: varchar (11), 必填, 收货人电话; 5. address: varchar (255), 必填, 详细收货地址; 6. is_default: tinyint (1), 必填, 默认 0, 是否默认地址
cart (购物车表)	存储用户购物车数据	cart_id	user_id、goods_id	1. cart_id: int (11), 必填, 自增, 主键, 购物车唯一标识; 2. user_id: int (11), 必填, 外键, 关联用户表; 3. goods_id: int (11), 必填, 外键, 关联商品表; 4. goods_num: int (11), 必填, 默认 1, 商品购买数量; 5. create_time: datetime, 必填, 默认当前时间, 添加时间
order (订单表)	存储用户团购订单	order_id	user_id、merchant_id、goods_id、address_id	1. order_id: int (11), 必填, 自增, 主键, 订单唯一标识; 2. order_no: varchar (50), 必填, 唯一, 订单编号; 3. user_id: int (11), 必填, 外键, 关联用户表; 4. merchant_id: int (11), 必填, 外键, 关联商家表; 5. goods_id: int (11), 必填, 外键, 关联商品表; 6. address_id: int (11), 必填, 外键, 关联地址表; 7. total_price: decimal (10,2), 必填, 订单总价; 8. order_status: varchar (20), 必填, 默认待付款, 订单状态; 9. create_time: datetime, 必填, 默认当前时间, 下单时间
collect (收藏表)	存储用户商品收藏	collect_id	user_id、goods_id	1. collect_id: int (11), 必填, 自增, 主键, 收藏唯一标识; 2. user_id: int (11), 必填, 外键, 关联用户表; 3. goods_id: int (11), 必填, 外键, 关联商品表; 4. create_time: datetime, 必填, 默认当前时间, 收藏时间

comment (评论表)	存储商品评价信息	comment_id	user_id、goods_id	1. comment_id: int (11), 必填, 自增, 主键, 评论唯一标识; 2. user_id: int (11), 必填, 外键, 关联用户表; 3. goods_id: int (11), 必填, 外键, 关联商品表; 4. content: text, 必填, 评论内容; 5. create_time: datetime, 必填, 默认当前时间, 评论时间
---------------	----------	------------	------------------	--

表 5-1 数据表设计汇总

5.4 索引设计

表名	索引字段	索引类型	设计目的
user	username	唯一索引	提升用户名查询、登录验证效率
user	phone	唯一索引	提升手机号登录、用户信息检索效率
merchant	merchant_account	唯一索引	提升商家账号登录、信息查询效率
merchant	phone	唯一索引	提升商家手机号检索、账号校验效率
category	category_name	唯一索引	提升商品分类查询、重复校验效率
goods	goods_name	普通索引	提升商品名称搜索、模糊查询效率
goods	merchant_id	普通索引	提升商家查询自身商品列表效率
goods	category_id	普通索引	提升商品分类筛选、列表查询效率
address	user_id	普通索引	提升用户查询个人收货地址效率
cart	user_id, goods_id	联合唯一索引	防止用户重复添加商品, 快速查询购物车数据
order	order_no	唯一索引	提升订单编号精准查询、订单校验效率
order	user_id	普通索引	提升用户查询个人订单列表效率
order	merchant_id	普通索引	提升商家查询本店订单数据效率
collect	user_id, goods_id	联合唯一索引	防止用户重复收藏商品, 快速查询收藏列表
comment	goods_id	普通索引	提升商品详情页查询评论数据效率

表 5-2 索引设计汇总

5.5 数据库脚本

遵循 MySQL 8.0 语法规则, 基于前文数据表设计编写, 包含全部核心表创建语句, 支持直接在 Navicat、SQLyog 中执行, 注释清晰、约束完整:

Sql

-- 创建用户表

```
CREATE TABLE `user` (  
  `user_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '用户唯一标识',  
  `username` varchar(50) NOT NULL COMMENT '用户名',  
  `password` varchar(100) NOT NULL COMMENT '加密存储的用户密码',  
  `role` varchar(20) NOT NULL DEFAULT '普通用户' COMMENT '用户角色（普通用户/管理员）',  
  `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP  
  COMMENT '注册时间',  
  `name` varchar(20) NOT NULL COMMENT '用户真实姓名',  
  `phone` varchar(11) NOT NULL COMMENT '用户手机号',  
  `email` varchar(50) DEFAULT NULL COMMENT '用户邮箱',  
  `avatar` varchar(255) DEFAULT NULL COMMENT '用户头像',  
  `gender` varchar(4) DEFAULT '未知' COMMENT '用户性别',  
  PRIMARY KEY (`user_id`),  
  UNIQUE KEY `idx_username` (`username`),  
  UNIQUE KEY `idx_phone` (`phone`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
```

-- 创建商家表

```
CREATE TABLE `merchant` (  
  `merchant_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '商家唯一标识',  
  `merchant_account` varchar(50) NOT NULL COMMENT '商家登录账号',  
  `password` varchar(100) NOT NULL COMMENT '加密存储的商家密码',  
  `merchant_name` varchar(50) NOT NULL COMMENT '商铺名称',  
  `phone` varchar(11) NOT NULL COMMENT '商家联系电话',
```

```

`address` varchar(255) NOT NULL COMMENT '商铺地址',
`status` tinyint(1) NOT NULL DEFAULT '0' COMMENT '审核状态 (0 待审核/1
已通过) ',
`create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP
COMMENT '入驻时间',
PRIMARY KEY (`merchant_id`),
UNIQUE KEY `idx_merchant_account` (`merchant_account`),
UNIQUE KEY `idx_merchant_phone` (`phone`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商家表';

-- 创建商品分类表
CREATE TABLE `category` (
`category_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '分类唯一
标识',
`category_name` varchar(30) NOT NULL COMMENT '分类名称',
`sort` int(11) NOT NULL DEFAULT '0' COMMENT '分类排序权重',
`create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP
COMMENT '创建时间',
PRIMARY KEY (`category_id`),
UNIQUE KEY `idx_category_name` (`category_name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品分类表';

-- 创建商品表
CREATE TABLE `goods` (
`goods_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '商品唯一标
识',
`goods_name` varchar(100) NOT NULL COMMENT '商品名称',
`category_id` int(11) NOT NULL COMMENT '关联分类 ID',

```

```

`merchant_id` int(11) NOT NULL COMMENT '关联商家 ID',
`price` decimal(10,2) NOT NULL COMMENT '商品原价',
`group_price` decimal(10,2) NOT NULL COMMENT '拼团价格',
`stock` int(11) NOT NULL DEFAULT '0' COMMENT '商品库存',
`limit_num` int(11) NOT NULL DEFAULT '1' COMMENT '单人限购数量',
`image` varchar(255) NOT NULL COMMENT '商品图片',
`status` tinyint(1) NOT NULL DEFAULT '0' COMMENT '上架状态 (0 待审核/1
已上架/2 已驳回) ', PRIMARY KEY (`goods_id`),
KEY `idx_goods_name` (`goods_name`),
KEY `idx_merchant_id` (`merchant_id`),
KEY `idx_category_id` (`category_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品表';

-- 创建收货地址表
CREATE TABLE `address` (
`address_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '地址唯一标识',
`user_id` int(11) NOT NULL COMMENT '关联用户 ID',
`receiver` varchar(20) NOT NULL COMMENT '收货人姓名',
`phone` varchar(11) NOT NULL COMMENT '收货人电话',
`address` varchar(255) NOT NULL COMMENT '详细收货地址',
`is_default` tinyint(1) NOT NULL DEFAULT '0' COMMENT '是否默认地址 (0
否/1 是) ',
PRIMARY KEY (`address_id`),
KEY `idx_user_id` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='收货地址表';

-- 创建购物车表

```

```

CREATE TABLE `cart` (
  `cart_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '购物车唯一标识',
  `user_id` int(11) NOT NULL COMMENT '关联用户 ID',
  `goods_id` int(11) NOT NULL COMMENT '关联商品 ID',
  `goods_num` int(11) NOT NULL DEFAULT '1' COMMENT '商品购买数量',
  `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '添加时间',
  PRIMARY KEY (`cart_id`),
  UNIQUE KEY `idx_user_goods` (`user_id`, `goods_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='购物车表';

-- 创建订单表
CREATE TABLE `order` (
  `order_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '订单唯一标识',
  `order_no` varchar(50) NOT NULL COMMENT '订单编号',
  `user_id` int(11) NOT NULL COMMENT '关联用户 ID',
  `merchant_id` int(11) NOT NULL COMMENT '关联商家 ID',
  `goods_id` int(11) NOT NULL COMMENT '关联商品 ID',
  `address_id` int(11) NOT NULL COMMENT '关联地址 ID',
  `total_price` decimal(10,2) NOT NULL COMMENT '订单总价',
  `order_status` varchar(20) NOT NULL DEFAULT '待付款' COMMENT '订单状态',
  `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '下单时间',
  PRIMARY KEY (`order_id`),
  UNIQUE KEY `idx_order_no` (`order_no`),

```



```

KEY `idx_order_user_id` (`user_id`),
KEY `idx_order_merchant_id` (`merchant_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='订单表';

-- 创建收藏表
CREATE TABLE `collect` (
`collect_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '收藏唯一标识',
`user_id` int(11) NOT NULL COMMENT '关联用户 ID',
`goods_id` int(11) NOT NULL COMMENT '关联商品 ID',
`create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '收藏时间',
PRIMARY KEY (`collect_id`),
UNIQUE KEY `idx_user_collect` (`user_id`,`goods_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='收藏表';

-- 创建评论表
CREATE TABLE `comment` (
`comment_id` int(11) NOT NULL AUTO_INCREMENT COMMENT '评论唯一标识',
`user_id` int(11) NOT NULL COMMENT '关联用户 ID',
`goods_id` int(11) NOT NULL COMMENT '关联商品 ID',
`content` text NOT NULL COMMENT '评论内容',
`create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '评论时间',
PRIMARY KEY (`comment_id`),
KEY `idx_comment_goods_id` (`goods_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='评论表';

```

6. 接口设计

6.1 接口设计原则

1. 接口命名规范、语义清晰，便于前后端开发人员理解；
2. 接口参数完整、格式统一，明确必填项与非必填项；
3. 接口返回数据格式统一，包含状态码、提示信息、核心数据；
4. 实现接口权限控制，不同角色只能访问对应接口；
5. 接口具有可扩展性，便于后续功能迭代。

6.2 接口通用格式

6.2.1 请求格式

请求方式：GET、POST、PUT、DELETE（查询用 GET，新增用 POST，修改用 PUT，删除用 DELETE）

请求头：Content-Type: application/json（前后端分离项目）

请求参数：GET 使用路径参数 / URL 参数，POST/PUT 使用请求体（body）参数

6.2.2 返回格式

```
json
{
  "code": 200, // 状态码：200 成功，400 参数错误，401 未登录，403 无权限，
500 服务器错误
  "msg": "操作成功", // 提示信息
  "data": { // 核心返回数据（无数据时可省略）
    // 具体数据内容
  }
}
```

6.3 核心接口详细设计

6.3.1 用户管理模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
用户注册	POST	/api/user/register	username (必填, string) : 用户名 password (必填, string) : 密码 code (必填, string) : 验证码	code:200, msg:"注册成功" data: {user_id: 1, username: "xxx"}	普通用户
用户登录	POST	/api/user/login	username (必填, string) : 用户名 password (必填, string) : 密码	code:200, msg:"登录成功" data: {token: "xxx", role: "普通用户"}	所有用户
获取用户信息	GET	/api/user/info	token (请求头)	code:200, msg:"查询成功" data: {user_id:1, username:"xxx", phone:"xxx"}	登录用户

修改用户资料	PUT	/api/user/update	username (string) : 用户名 phone (string) : 手机号 email (string) : 邮箱	code:200, msg:"修改成功"	登录用户
退出登录	POST	/api/user/logout	token (请求头)	code:200, msg:"退出成功"	登录用户
修改密码	PUT	/api/user/update-pwd	old_pwd (string) : 旧密码 new_pwd (string) : 新密码	code:200, msg:"密码修改成功"	登录用户

表 6-1 用户管理模块接口

6.3.2 商品管理模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
获取商品列表	GET	/api/goods/list	category_id (int) : 分类 ID keyword (string) : 关键词 page (int) : 页码 size (int) : 每页条数	code:200, msg:"查询成功" data: {total:100, list:[商品对象数组]}	所有用户
获取商品详情	GET	/api/goods/detail/{id}	id (路径参数) : 商品 ID	code:200, msg:"查询成功" data: 商品详细信息	所有用户

商家新增商品	POST	/api/goods/add	name、price、stock、content、category_id、image	code:200, msg:"新增成功"	商家
商家编辑商品	PUT	/api/goods/update	goods_id (必填)、其他商品字段	code:200, msg:"修改成功"	商家
商品上下架	PUT	/api/goods/change-status	goods_id、status (0 下架 1 上架)	code:200, msg:"状态修改成功"	商家

表 6-2 商品管理模块接口

6.3.3 购物车模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
添加商品到购物车	POST	/api/cart/add	goods_id、num (数量)	code:200, msg:"添加成功"	登录用户
获取购物车列表	GET	/api/cart/list	无 (token)	code:200, msg:"查询成功" data: 购物车列表	登录用户
修改购物车数量	PUT	/api/cart/update-num	cart_id、num	code:200, msg:"修改成功"	登录用户
删除购物车商品	DELETE	/api/cart/delete/{id}	id: 购物车项 ID	code:200, msg:"删除成功"	登录用户

表 6-3 购物车模块接口

6.3.4 订单交易模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
创建订单	POST	/api/order/create	cart_ids (数组)、 address_id	code:200, msg:"订单创建成功"data:{order_id:"xxx"}	登录用户
获取订单列表	GET	/api/order/list	status: 订单状态	code:200, msg:"查询成功"data: 订单列表	登录用户
订单支付	POST	/api/order/pay	order_id、 pay_type	code:200, msg:"支付成功"	登录用户
确认收货	PUT	/api/order/ receive/{id}	order_id	code:200, msg:"确认收货成功"	登录用户
取消订单	PUT	/api/order/cancel/ {id}	order_id	code:200, msg:"取消成功"	登录用户

表 6-4 订单交易模块接口

6.3.5 拼团模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
发起拼团	POST	/api/group/create	goods_id、 num	code:200, msg:"发起成功"	登录用户
参与拼团	POST	/api/group/join	group_id、 goods_id	code:200, msg:"参与成功"	登录用户
获取拼团列表	GET	/api/group/list	goods_id	code:200, msg:"查询成功"data:	所有用户

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
				拼团列表	
获取拼团详情	GET	/api/group/detail/{id}	group_id	code:200, msg:"查询成功"data: 拼团详情 + 成员	所有用户

表 6-5 拼团模块接口

6.3.6 收货地址模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
新增地址	POST	/api/address/add	receiver、phone、 address、is_default	code:200, msg:"新增成功"	登录用户
地址列表	GET	/api/address/list	无	code:200, msg:"查询成功"data: 地址列表	登录用户
编辑地址	PUT	/api/address/ update	address_id + 地址信息	code:200, msg:"修改成功"	登录用户
删除地址	DELETE	/api/address/ delete/{id}	address_id	code:200, msg:"删除成功"	登录用户

表 6-6 收货地址模块接口

6.3.7 商家管理模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
商家订单列表	GET	/api/merchant/order/list	status、keyword	code:200, msg:"查询成功"data: 订单列表	商家
商家订单发货	PUT	/api/merchant/order/send	order_id、 express_no、 express_name	code:200, msg:"发货成功"	商家
商家数据统计	GET	/api/merchant/statistics	无	code:200, msg:"查询成功"data: 统计数据	商家

表 6-7 商家管理模块接口

6.3.8 管理员系统模块接口

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
管理员登录	POST	/api/admin/login	username、password	code:200, msg:"登录成功"data:token	管理员
用户管理列表	GET	/api/admin/user/list	keyword、status	code:200, msg:"查询成功"	管理员
商品分类管理	POST/ PUT/ DELETE	/api/admin/category	分类信息	code:200, msg:"操作成功"	管理员
公告发布	POST	/api/admin/	title、content、sort	code:200, msg:"发布成功"	管理员

接口名称	请求方式	接口地址	请求参数	返回数据	操作角色
		notice/publish			

表 6-8 管理员系统模块接口

7. 界面详细设计

7.1 界面设计原则

1. 与需求文档界面需求一致，风格统一、布局合理、色彩协调；
2. 操作流程简洁，符合用户使用习惯，减少操作步骤；
3. 界面元素（按钮、输入框、菜单）命名规范，位置固定，便于用户识别；
4. 适配常用浏览器，无兼容性问题，响应式设计（可选）。

7.2 核心界面详细设计（按页面分类）

页面名称	对应模块	界面布局	核心元素	交互逻辑
登录页面	用户管理模块	居中布局，顶部标题，中间表单区域，底部注册 / 忘记密码入口	账号输入框、密码输入框、登录按钮、验证码输入框、注册链接	输入验证、登录提交、跳转注册
首页	核心功能模块	顶部导航栏，中间内容展示区，底部信息栏	功能入口按钮、热门信息展示、通知公告、用户头像 / 退出按钮	点击跳转、信息刷新、退出登录
用户注册页面	用户管理模块	居中表单布局，顶部标题，中间注册表单，底部返回登录入口	用户名、密码、确认密码、手机号、邮箱、头像上传、注册按钮、返回登录链接	字段校验、用户名查重、注册提交、跳转登录
商品列表页面	商品管理模块	顶部搜索 + 分类栏，中间商品瀑布流，底部分页	搜索框、商品分类、商品卡片、排序按钮、分页控件	分类筛选、关键词搜索、排序切换、点击进入商品详情
商品详情页面	商品管理模块	顶部商品图区，中部详情信息，底部操作栏	商品图片、价格 / 团购价、库存、单限、拼团人数、加入购物车、立即购买、发起拼团、收藏、评论区	数量选择、加入购物车、发起 / 参与拼团、收藏、查看评论、发表评论
购物车页面	订单交易模块	顶部标题，中间商品列表，底部结算栏	商品项、数量修改、删除、全选、合计金额、结算按钮	数量修改、删除商品、勾选结算、提交订单

订单确认页面	订单交易模块	顶部地址栏，中间商品清单，底部支付栏	收货地址、商品清单、订单金额、支付方式、提交订单按钮	地址选择、订单校验、库存扣减、生成订单、跳转支付
拼团详情页面	拼团功能模块	顶部拼团信息，中部参团列表，底部操作区	拼团进度、剩余时间、参团用户、邀请好友、我要参团按钮	查看拼团状态、参与拼团、倒计时自动关闭未成团订单
个人中心页面	用户管理模块	顶部用户信息，中部功能菜单，底部退出登录	头像、用户名、我的订单、我的收藏、我的地址、修改资料、退出登录	菜单跳转、资料修改、退出登录
我的订单页面	订单交易模块	顶部状态标签栏，中间订单列表，底部无数据提示	全部 / 待付款 / 待发货 / 待收货 / 已完成、订单卡片、取消订单、确认收货、查看物流	状态切换、订单详情、取消未付款订单、确认收货、查看物流
收货地址管理页面	用户管理模块	顶部标题，中间地址列表，底部新增地址按钮	地址项、设为默认、编辑、删除、新增地址表单	新增 / 编辑 / 删除地址、设置默认、下单自动选用
商家后台首页	商家管理模块	左侧菜单栏，右侧数据概览，顶部商家信息	订单统计、商品统计、待处理订单、功能菜单（商品、订单、公告）	菜单跳转、数据查看、快速处理订单
商家商品管理页面	商家管理模块	顶部操作栏，中间商品列表，底部分页	新增商品、上架 / 下架、编辑、删除、商品搜索、分页	新增商品、编辑信息、上下架、删除商品、筛选查询
商家订单管理页面	商家管理模块	顶部状态筛选，中间订单列表，底部操作	订单查询、发货、填写物流、取消订单、订单详情	筛选订单、查看详情、确认发货、更新物流、取消未付款订单
管理员后台首页	系统管理模块	左侧全局菜单，右侧数据看板，顶部管理员信息	用户统计、商家统计、商品统计、系统快捷入口	全局菜单跳转、数据概览、快速审核
管理员用户管理页面	系统管理模块	顶部搜索栏，中间用户列表，底部操作	查询、新增、编辑、删除、重置密码、分页	用户检索、信息编辑、账号禁用、批量操作
管理员商品分类管理页	系统管理模块	顶部操作栏，中间分类列表，底部分页	新增分类、编辑、删除、分类排序	新增分类、修改名称、删除空分类、排

面				序调整
活动公告管理页面	系统管理模块	顶部发布按钮，中间公告列表，底部操作	发布公告、编辑、删除、置顶、预览	发布 / 编辑公告、置顶展示、前台同步更新

表 7-1 核心界面详细设计汇总

(附：核心界面原型图粘贴处，每个页面粘贴对应原型图，标注页面名称与核心功能)

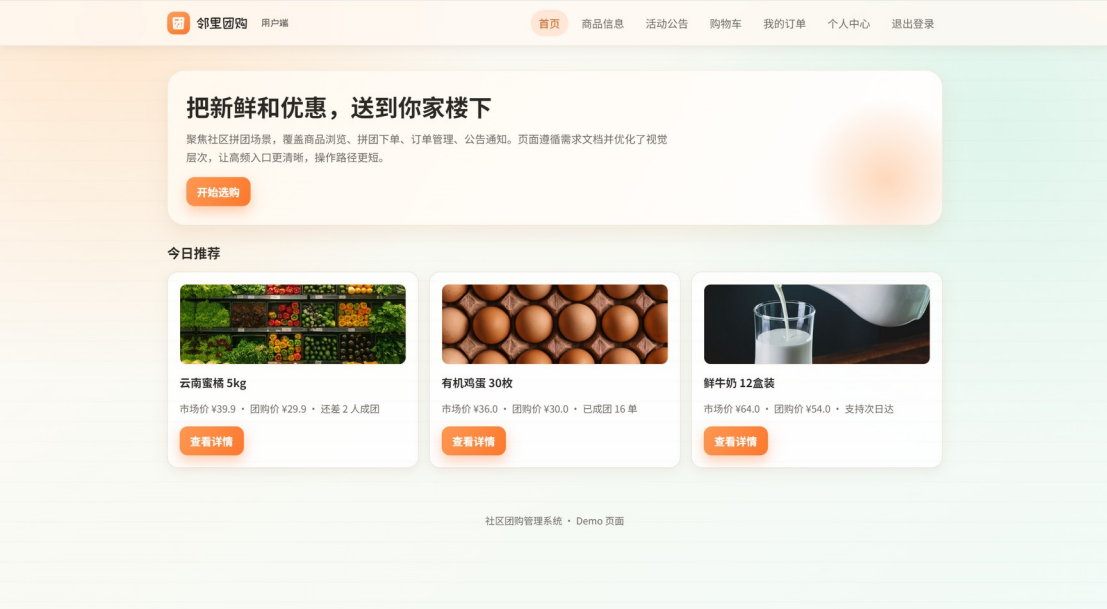


图 7-1 系统首页界面

点击用户注册，用户通过注册页面填写用户名、密码、确认密码、姓名、头像、性别、邮箱、手机等信息，并点击注册操作，如图 7-2 所示。

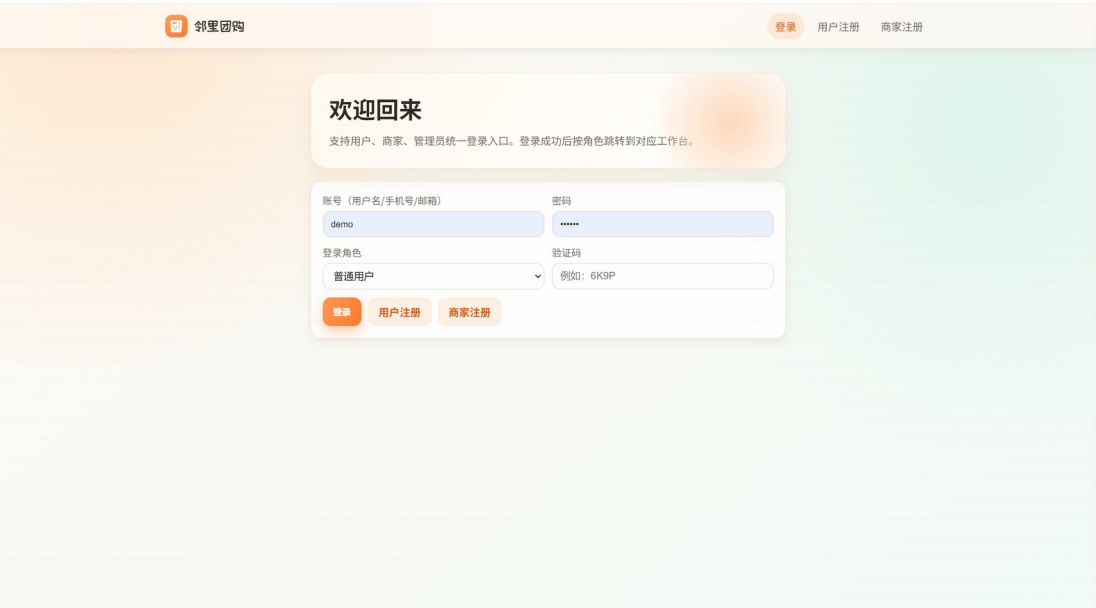


图 7-2 用户注册界面图

管理员如果要登录到系统后台，首先得先通过输入用户名、密码、选择角色进行

登录的操作。管理员登录界面如图 7-3 所示。

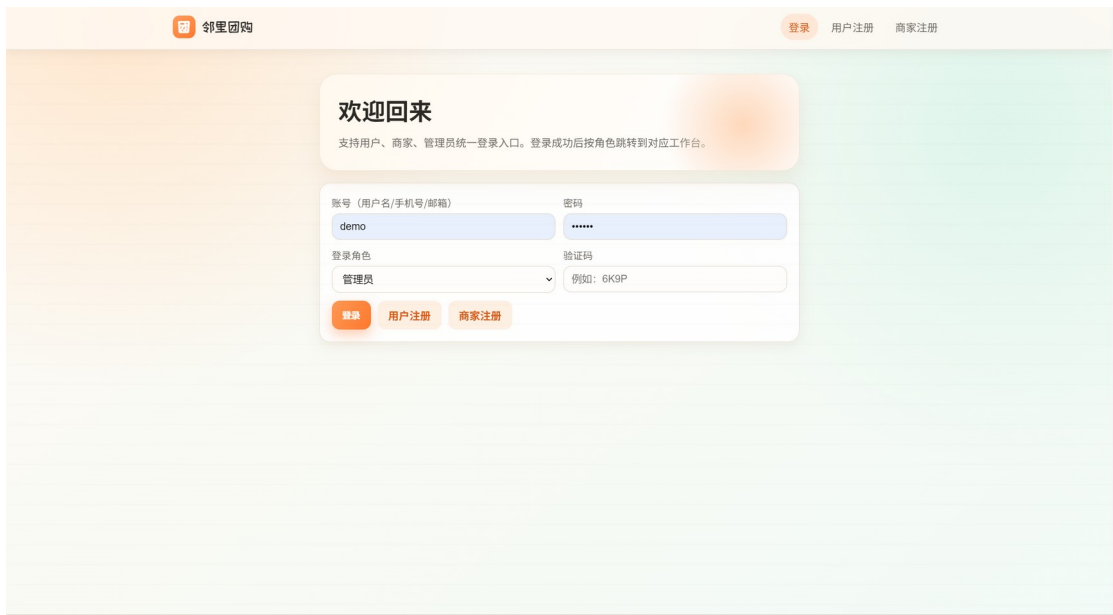


图 7-3 管理员登录界面

管理员登录到系统后台后，可以对系统首页、个人中心、用户管理、商家管理、商品分类管理、商品信息管理、活动公告管理、系统管理等功能模块进行管理；如图 7-4 所示



图 7-4 管理员功能界面

管理员点击用户管理，在用户管理页面，对用户名、姓名、头像、性别、邮箱、手机等内容，进行查询或者新增、删除用户信息等操作，如图 7-5 所示。

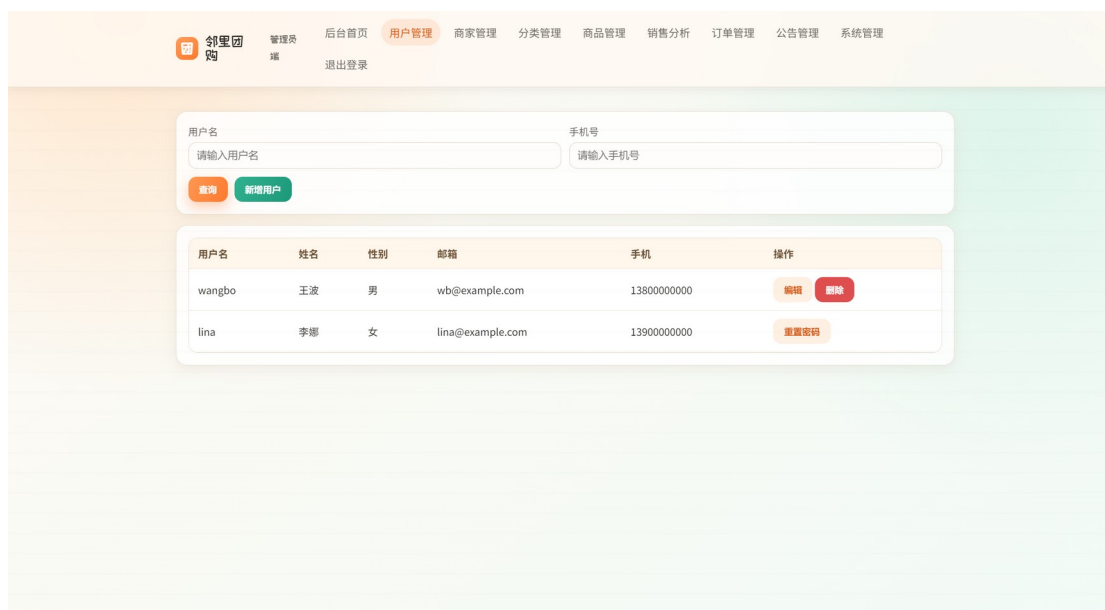


图 7-5 用户管理界面图

管理员点击商家管理，在商家管理页面，对账号、商家姓名、性别、头像、联系电话等内容，进行查询或者新增、删除商家信息等操作，如图 6-6 所示。

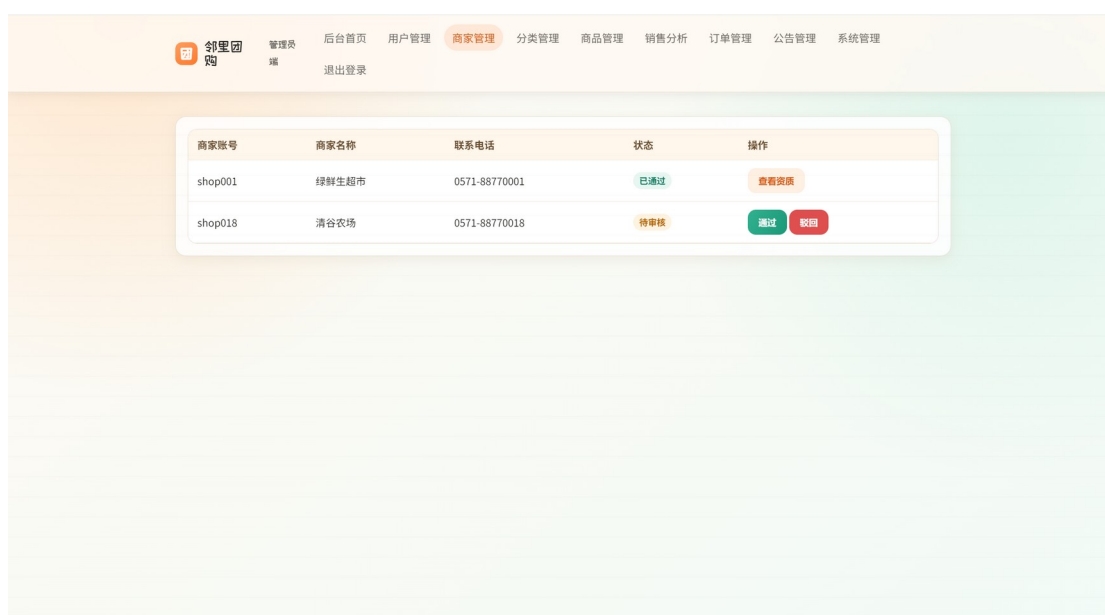


图 7-6 商家管理界面图

管理员点击商品分类管理，在商品分类管理页面，对商品分类等内容，进行查询或者删除商品分类等操作，如图 7-7 所示。

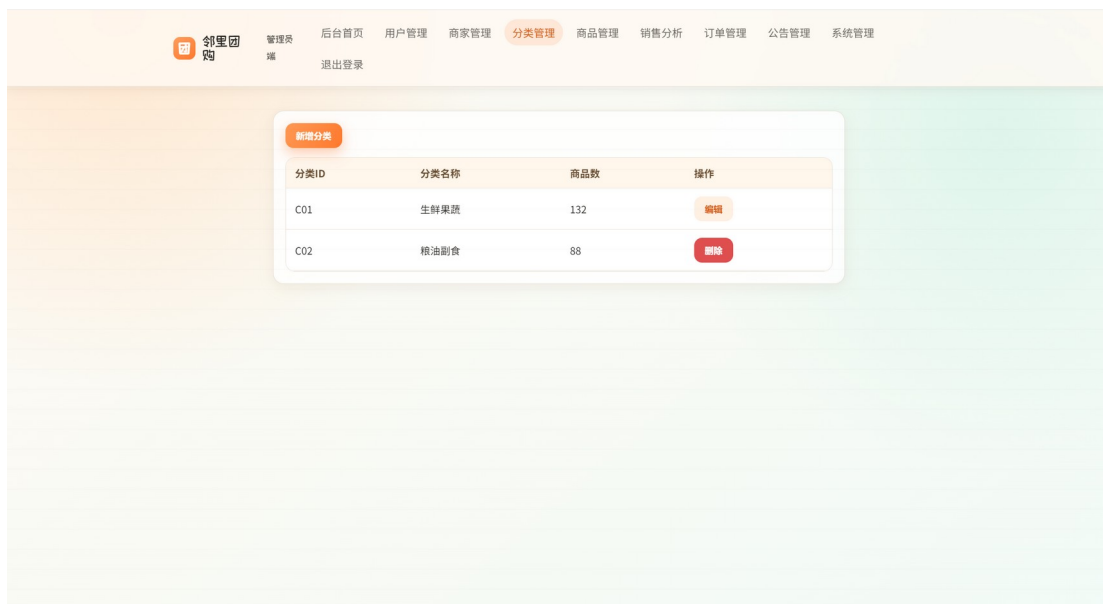


图 7-7 商品分类管理界面图

管理员点击活动公告管理，在活动公告管理页面，对商品名称、商品分类、品牌、图片、账号、上架日期、单限、库存、价格等内容，进行查询或者删除活动公告等操作，如图 7-8 所示。

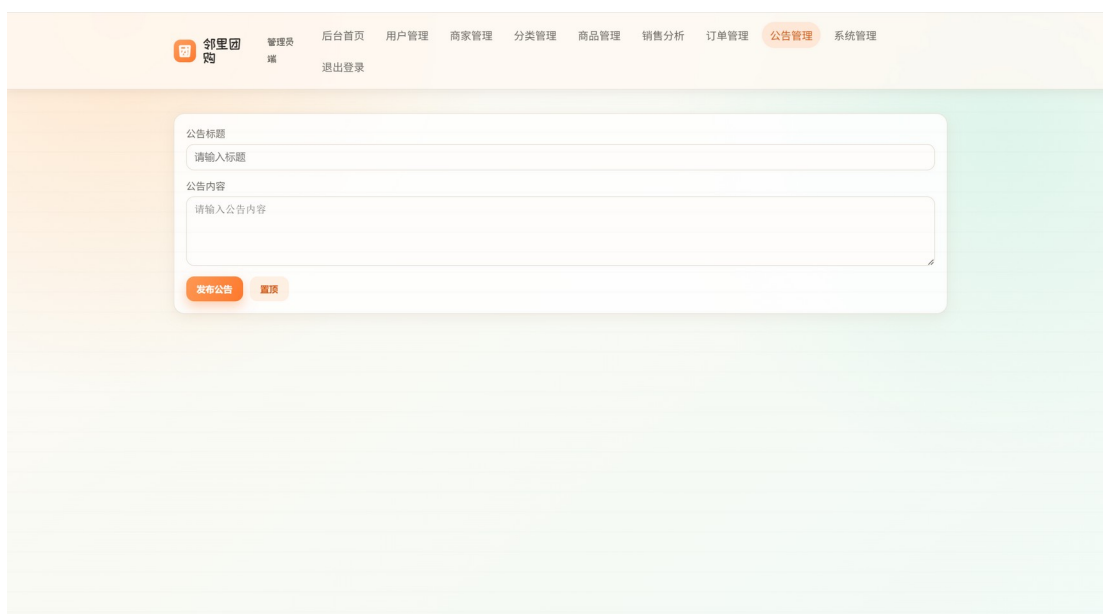


图 7-8 活动公告管理界面图

3.后台商家功能模块

商家注册、登录，商家通过注册、登录页面输入信息完成后，并点击注册或者登录操作，如图 7-9 所示。



图 7-9 商家注册、商家登录界面图

商家登录系统后，可以对系统首页、个人中心、商品信息管理、订单管理等功能进行相应操作，如图 7-10 所示。



图 7-10 商家功能界面图

8. 开发规范

8.1 编码规范

1. 命名规范：类名采用帕斯卡命名法（如：UserController），方法名、变量名采用驼峰命名法（如：loginUser），常量名采用全大写+下划线（如：MAX_PAGE_SIZE）；
2. 代码格式：缩进统一（4 个空格），每行代码不超过 80 字符，注释清晰（类注释、方法注释、关键逻辑注释）；
3. 异常处理：统一捕获异常，返回规范提示信息，避免直接抛出异常；
4. 代码复用：提取公共方法、工具类，减少代码冗余，提升代码可维护性。

8.2 文档规范

1. 文档格式统一，字体、行距、标题层级与本模板保持一致；
2. 图表清晰，标注完整，流程图、E-R 图、原型图需贴合设计内容；
3. 语言简洁、逻辑清晰，避免冗余描述，确保文档可阅读、可参考。

8.3 版本控制规范

1. Git 提交规范：提交信息格式为“类型: 描述”（如：feat: 新增用户注册功能）；
2. 分支管理：主分支（main）用于存放最终可运行代码，开发分支（dev）用于日常开发，每个功能单独创建功能分支（feat-xxx）；
3. 每日提交代码，提交记录清晰，便于追溯与代码合并。

9. 附录

9.1 团队信息

小组名称： 组长：陈小庆 成员：王波，魏伊阳，杨洋（38）

9.2 其他说明

本系统为软件工程实训项目，受开发周期、技术栈与实训场景限制，存在部分未完善内容与设计难点，同时明确后续优化方向：

1. 未解决的设计问题

未集成第三方支付接口，仅采用模拟支付完成订单流程，无真实支付与退款逻辑；
未对接物流跟踪 API，仅支持手动填写物流单号，无法实现物流轨迹实时查询；
拼团有效期、自动退款、超时取消等规则未做细化设计，仅实现基础拼团逻辑；
无短信、站内信等消息通知功能，订单状态变更需用户主动登录查看。

2. 设计难点说明

拼团场景下库存并发扣减易出现超卖问题，需通过事务与锁机制保证数据一致性；

多角色（用户 / 商家 / 管理员）RBAC 权限隔离需严格控制接口访问，防止越权操作；
商品搜索、订单查询等高频操作需优化索引与 SQL，平衡查询效率与数据库压力。

3.后续优化方向

接入微信支付、支付宝，完善在线支付、退款、订单核销功能；
对接物流平台 API，实现物流信息自动同步与实时跟踪；
新增优惠券、积分、秒杀等营销功能，提升用户活跃度；
开发微信小程序版本，优化移动端体验，扩展使用场景；
增加数据可视化看板，为管理员提供销售、用户、商品等运营统计数据。